Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ДОМАШНЕЕ ЗАДАНИЕ №1

«Реализация алгоритма поиска с опечатками»

ДИСЦИПЛИНА: «Архитектура автоматизированных систем обработки информации и управления»

Выполнил: студент гр. ИУ5-22Б _____ (Маслов А. О.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Гапанюк Ю. Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

Москва, 2026 г.

Условия домашнего задания №1

- Домашнее задание реализуется в виде консольного приложения на языке программирования C#. Рекомендуется использовать операторы верхнего уровня.
- Пользователь вводит две строки, после чего программа выводит значение расстояния Левенштейна между этими строками.
- Указанные выше действия реализуются в виде вечного цикла. Для выхода из цикла в качестве первой строки вводится значение «exit».
- Для сдачи домашнего задания необходимо продемонстрировать работоспособность программы, ответить на вопросы и предоставить отчет, содержащий текст программы и результаты ее выполнения.

Основной код программы:

```
namespace ConsoleApp1
{
    internal class Program
    {
        public static int Distance(string str1Param, string str2Param)
        {
            if ((str1Param == null) || (str2Param == null)) return -1;
            int str1Len = str1Param.Length;
            int str2Len = str2Param.Length;

            if ((str1Len == 0) & (str2Len == 0)) return 0;
            if (str1Len == 0) return str2Len;
            if (str2Len == 0) return str1Len;

            string str1 = str1Param.ToUpper();
            string str2 = str2Param.ToUpper();

            int[,] matrix = new int[str1Len+1, str2Len+1];
            for (int i = 0; i <= str1Len; i++) matrix[i, 0] = i;
            for (int j = 0; j <= str2Len; j++) matrix[0, j] = j;
            for (int i = 1; i <= str1Len; i++)
            {
                for (int j = 1;j <= str2Len; j++)
                {
                    int SymbEqual = (
                        (str1.Substring(i-1, 1) == str2.Substring(j-1, 1)) ? 0 : 1);
                    int ins = matrix[i, j - 1] + 1;
                    int del = matrix[i - 1, j] + 1;
                    int rep = matrix[i - 1, j - 1] + SymbEqual;

                    matrix[i, j] = Math.Min(del, Math.Min(ins, rep));

                    if ((i > 1)  && (j > 1) &&
                        (str1.Substring((i - 1), 1) == str2.Substring((j - 2), 1)) &&
                        (str1.Substring((i - 2), 1) == str2.Substring((j-1), 1)))
                    {
                        matrix[i, j] = Math.Min(matrix[i, j], matrix[i - 2, j - 2] +
SymbEqual);
                    }
                }
            }
            return matrix[str1Len, str2Len];
        }
        public static void WriteDistance(string str1Param, string str2Param)
        {
            int a = Distance(str1Param, str2Param);
            Console.WriteLine($"{str1Param}, {str2Param} -> {a}");
        }
        private static void Main(string[] args)
        {
            Console.WriteLine("Введите 1 строку (для выхода напишите exit): ");
            string? s1 = Console.ReadLine();
```

```
            while ((s1 != null) && (s1 != "exit")) {
                Console.WriteLine("Введите 2 строку: ");
                string? s2 = Console.ReadLine();
                if (s2 == null) break;
                WriteDistance(s1, s2);
                Console.WriteLine("Введите 1 строку (для выхода напишите exit): ");
                s1 = Console.ReadLine();
            }
        }
    }
}
```

Результат выполнения программы:

```
C:\Users\Артём\source\repos\

Введите 1 строку (для выхода напишите exit):
привет
Введите 2 строку:
hello
привет, hello -> 6
Введите 1 строку (для выхода напишите exit):
пока
Введите 2 строку:
поак
пока, поак -> 1
Введите 1 строку (для выхода напишите exit):
арбуз
Введите 2 строку:
арбалет
арбуз, арбалет -> 4
Введите 1 строку (для выхода напишите exit):
exii
Введите 2 строку:
exit
exii, exit -> 1
Введите 1 строку (для выхода напишите exit):
exit
```

Ответы на контрольные вопросы:

1. ***Что такое расстояние Левенштейна?***

Расстоянием Левенштейна называется минимальное количество действий, необходимых для преобразования одного слова в другое. Основные операции, учитываемые при вычислении расстояния Левенштейна:

D (delete) — удаление символа

I (insert) — добавление символа

R (replace) — замена символа

M (match) — совпадение символов (не требует действий)

2. Чем отличается расстояние Левенштейна от расстояния Дамерау-Левенштейна?

Расстояние Дамерау-Левенштейна — это усовершенствованная версия расстояния Левенштейна, которая дополнительно учитывает операцию транспозиции (перестановки двух соседних символов) как единичное действие.

При использовании расстояния Дамерау-Левенштейна за единичное расстояние принимаются следующие действия:

I (insert) — добавление символа

D (delete) — удаление символа

R (replace) — замена символа

T (transposition) — перестановка двух соседних символов

3. Объясните алгоритм Вагнера-Фишера

Алгоритм Вагнера-Фишера использует динамическое программирование для вычисления расстояния Левенштейна.

Как работает:

1) Создается матрица размером (M+1) × (N+1), где M и N — длины строк.
2) Заполнение матрицы:
    a. Первая строка: 0,1,2,...,N (вставки)
    b. Первый столбец: 0,1,2,...,M (удаления)
    c. Для каждой ячейки D[i,j] считаем минимум из трех вариантов:
    d. D[i-1, j] + 1 — удаление
    e. D[i, j-1] + 1 — вставка
    f. D[i-1, j-1] + cost — замена (0 если символы совпадают, 1 если разные)

Дополнительно для Дамерау-Левенштейна:

3) Проверяем возможность транспозиции (перестановки соседних символов)
4) Если символы можно переставить, учитываем D[i-2, j-2] + cost
5) Результат — значение в правом нижнем углу матрицы.

Сложность: O(M × N) по времени и памяти.